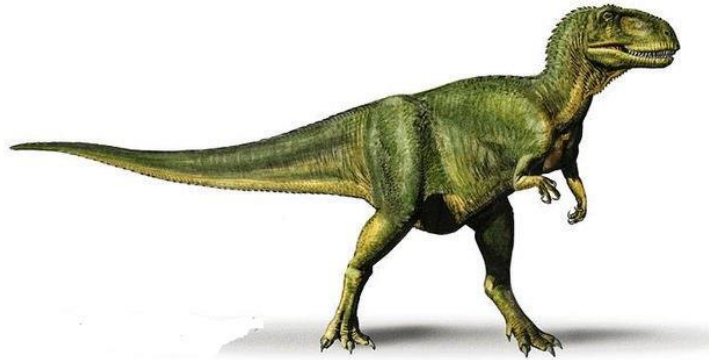




SISTEMAS OPERATIVOS Y REDES

COMUNICACIÓN INTERPROCESOS



Contenido

Comunicación Interprocesos (IPC)	2
Prologo	2
Objetivos de este resumen	2
Intro.....	2
Sistemas de memoria compartida	3
Sistemas de paso de mensajes.....	6

Comunicación Interprocesos (IPC)

Prologo

Bienvenidos al resumen adaptado del libro Fundamentos de Sistemas Operativos (Silberchtaiz-Galvin-Gagne) que acompaña la cátedra Sistemas Operativos y Redes de la carrera de Informática. Este documento tiene como objetivo proporcionar un acceso conciso y claro a los conceptos fundamentales presentados en el libro de texto principal, adaptado específicamente para satisfacer las necesidades y los objetivos del curso.

Los sistemas operativos son la columna vertebral de la informática moderna, y comprender su funcionamiento es esencial para cualquier estudiante de ciencias de la computación. Este resumen condensa los conceptos clave, los principios fundamentales y las técnicas avanzadas discutidas en el libro de texto original, proporcionando una guía clara y accesible para el estudio y la comprensión de los sistemas operativos.

A lo largo de este resumen, encontrarás una serie de capítulos que abordan temas importantes como la gestión de procesos, la gestión de memoria, la planificación de la CPU, entre otros. Cada capítulo se ha diseñado cuidadosamente para ofrecer una visión completa y bien estructurada de los conceptos presentados en el libro de texto, acompañado de ejemplos prácticos y explicaciones claras para facilitar el aprendizaje.

Espero que este resumen sea una herramienta valiosa para complementar tu estudio de sistemas operativos y te ayude a alcanzar tus objetivos. Te deseamos mucho éxito en tu cursada.

Lic. Mariano Vargas

Objetivos de este resumen

- Comprender los diversos mecanismos relacionados con los mecanismos de comunicación.
- Describir los mecanismos de comunicación en los sistemas cliente-servidor.

Intro

Los procesos que se ejecutan concurrentemente pueden ser procesos independientes o procesos cooperativos. Un proceso es independiente si no puede afectar o verse afectado por los restantes procesos que se ejecutan en el sistema. Cualquier proceso que no comparte datos con ningún otro proceso es un proceso independiente. Un proceso es cooperativo si puede afectar o verse afectado por los demás procesos que se ejecutan en el sistema. Evidentemente, cualquier proceso que comparte datos con otros procesos es un proceso cooperativo.

Hay varias razones para proporcionar un entorno que permita la cooperación entre procesos:

- **Compartir información.** Dado que varios usuarios pueden estar interesados en la misma información (por ejemplo, un archivo compartido), debemos proporcionar un entorno que permita el acceso concurrente a dicha información.

- **Acelerar los cálculos.** Si deseamos que una determinada tarea se ejecute rápidamente, debemos dividirla en subtareas, ejecutándose cada una de ellas en paralelo con las demás. Observe que tal aceleración sólo se puede conseguir si la computadora tiene múltiples elementos de procesamiento, como por ejemplos varias CPU o varios canales de E/S.
- **Modularidad.** Podemos querer construir el sistema de forma modular, dividiendo las funciones del sistema en diferentes procesos o hebras.
- **Conveniencia.** Incluso un solo usuario puede querer trabajar en muchas tareas al mismo tiempo. Por ejemplo, un usuario puede estar editando, imprimiendo y compilando en paralelo.

La cooperación entre procesos requiere mecanismos de comunicación interprocesos (IPC, interprocess communication) que les permitan intercambiar datos e información. Existen dos modelos fundamentales de comunicación interprocesos:

- memoria compartida y
- paso de mensajes.

En el modelo de memoria compartida, se establece una región de la memoria para que sea compartida por los procesos cooperativos. De este modo, los procesos pueden intercambiar información leyendo y escribiendo datos en la zona compartida. En el modelo de paso de mensajes, la comunicación tiene lugar mediante el intercambio de mensajes entre los procesos cooperativos.

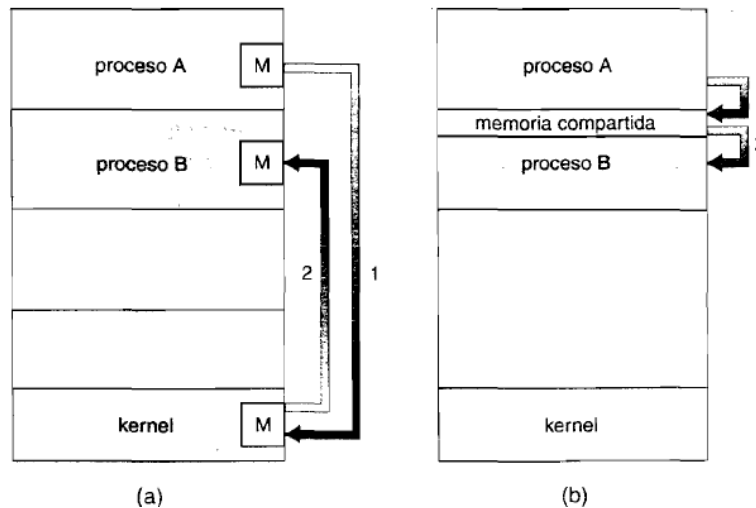
Los dos modelos son bastante comunes en los distintos sistemas operativos y muchos sistemas implementan ambos. El paso de mensajes resulta útil para intercambiar pequeñas cantidades de datos, ya que no existe la necesidad de evitar conflictos. El paso de mensajes también es más fácil de implementar que el modelo de memoria compartida como mecanismo de comunicación entre computadoras. La memoria compartida permite una velocidad máxima y una mejor comunicación, ya que puede realizarse a velocidades de memoria cuando se hace en una misma computadora. La memoria compartida es más rápida que el paso de mensajes, ya que este último método se implementa normalmente usando llamadas al sistema y, por tanto, requiere que intervenga el kernel, lo que consume más tiempo. Por el contrario, en los sistemas de memoria compartida, las llamadas al sistema sólo son necesarias para establecer las zonas de memoria compartida. Una vez establecida la memoria compartida, todos los accesos se tratan como accesos a memoria rutinarios y no se precisa la ayuda del kernel.

Sistemas de memoria compartida

La comunicación interprocesos que emplea memoria compartida requiere que los procesos que se estén comunicando establezcan una región de memoria compartida. Normalmente, una región de memoria compartida reside en el espacio de direcciones del proceso que crea el segmento de memoria compartida. Otros procesos que deseen comunicarse usando este segmento de memoria compartida deben conectarse a su espacio de direcciones. Recuerde que, habitualmente, el sistema operativo intenta evitar que un proceso acceda a la memoria de otro proceso. La memoria compartida requiere que dos o más procesos acuerden eliminar esta restricción. Entonces podrán intercambiar información leyendo y escribiendo datos en las áreas compartidas. El formato de los datos y su ubicación están

determinados por estos procesos, y no se encuentran bajo el control del sistema operativo. Los procesos también son responsables de verificar que no escriben en la misma posición simultáneamente.

Para ilustrar el concepto de procesos cooperativos, consideremos el problema del productor-consumidor, el cual es un paradigma comúnmente utilizado para los procesos cooperativos. Un proceso productor genera información que consume un proceso consumidor. Por ejemplo, un compilador puede generar código ensamblado, que consume un ensamblador. El ensamblador, a su vez, puede generar módulos objeto, que consume el cargador. El problema del productor-consumidor también proporciona una metáfora muy útil para el paradigma cliente-servidor.



Modelos de comunicación, (a) Paso de mensajes, (b) Memoria compartida

Generalmente, pensamos en un servidor como en un productor y en un cliente como en un consumidor. Por ejemplo, un servidor web produce (es decir, proporciona) archivos HTML e imágenes, que consume (es decir, lee) el explorador web cliente que solicita el recurso.

Una solución para el problema del productor-consumidor es utilizar mecanismos de memoria compartida. Para permitir que los procesos productor y consumidor se ejecuten de forma concurrente, debemos tener disponible un búfer de elementos que pueda rellenar el productor y vaciar el consumidor. Este búfer residirá en una región de memoria que será compartida por ambos procesos, consumidor y productor. Un productor puede generar un elemento mientras que el consumidor consume otro. El productor y el consumidor deben estar sincronizados, de modo que el consumidor no intente consumir un elemento que todavía no haya sido producido. Pueden emplearse dos tipos de búferes. El sistema de búfer no limitado no pone límites al tamaño de esa memoria compartida. El consumidor puede tener que esperar para obtener elementos nuevos, pero el productor siempre puede generar nuevos elementos. El sistema de búfer limitado establece un tamaño de búfer fijo. En este caso, el consumidor tiene que esperar si el búfer está vacío y el productor tiene que esperar si el búfer está lleno.

Veamos más en detalle cómo puede emplearse un búfer limitado para permitir que los procesos compartan la memoria. Las siguientes variables residen en una zona de la memoria compartida por los procesos consumidor y productor:

```
#define BUFFER_SIZE 10

typedef struct {
    item;
    item buffer[BUFFER_SIZE] ;
    int in = 0;
    int out = 0;
```

El búfer compartido se implementa como una matriz circular con dos punteros lógicos: in y out. La variable in apunta a la siguiente posición libre en el búfer; out apunta a la primera posición ocupada del búfer. El búfer está vacío cuando $in == out$; el búfer está lleno cuando $((in + 1) \% BUFFER_SIZE) == out$.

El proceso productor tiene una variable local, nextProduced, en la que se almacena el elemento nuevo que se va a generar. El proceso consumidor tiene una variable local, nextConsumed, en la que se almacena el elemento que se va a consumir.

Este esquema permite tener como máximo $BUFFER_SIZE - 1$ elementos en el búfer al mismo tiempo. Dejamos como ejercicio para el lector proporcionar una solución en la que $BUFFER_SIZE$ elementos puedan estar en el búfer al mismo tiempo.

Un problema del que no se ocupa este ejemplo es la situación en la que tanto el proceso productor como el consumidor intentan acceder al búfer compartido de forma concurrente.

```
item nextProduced;

while (true) {
    * produce e inserta "un elemenco en nextProduced*/
    vrhile ( (in-rl) % BUFFER_SIZE) == out)
    /*no hacer nada*/
    ouf fer [ir.. ^nextProduced;
    m = (m-li % BUFFER_3IZE;
    item nextConsumed;
    while (true) {
        while (in == out)
            /*no hacer nada*/
        nextConsumed = buffer[out];
        out = (out + 1) % BUFFER_3IZE;
```

```
/* consume el elemento almacenado en nextConsumed */  
}
```

El proceso productor.

Sistemas de paso de mensajes

Hemos visto cómo pueden comunicarse procesos cooperativos en un entorno de memoria compartida. El esquema requiere que dichos procesos compartan una zona de la memoria y que el programador de la aplicación escriba explícitamente el código para acceder y manipular la memoria compartida. Otra forma de conseguir el mismo efecto es que el sistema operativo proporcione los medios para que los procesos cooperativos se comuniquen entre sí a través de una facilidad de paso de mensajes.

El paso de mensajes proporciona un mecanismo que permite a los procesos comunicarse y sincronizar sus acciones sin compartir el mismo espacio de direcciones, y es especialmente útil en un entorno distribuido, en el que los procesos que se comunican pueden residir en diferentes computadoras conectadas en red. Por ejemplo, un programa de chat-utilizado en la Internet podría diseñarse de modo que los participantes en la conversación se comunicaran entre sí intercambiando mensajes.

Una facilidad de paso de mensajes proporciona al menos dos operaciones: envío de mensajes (send) y recepción de mensajes (receive). Los mensajes enviados por un proceso pueden tener un tamaño fijo o variable. Si sólo se pueden enviar mensajes de tamaño fijo, la implementación en el nivel de sistema es directa. Sin embargo, esta restricción hace que la tarea de programación sea más complicada. Por el contrario, los mensajes de tamaño variable requieren una implementación más compleja en el nivel de sistema, pero la tarea de programación es más sencilla. Este es un tipo de compromiso que se encuentra muy habitualmente en el diseño de sistemas operativos.

Si los procesos P y Q desean comunicarse, tienen que enviarse mensajes entre sí; debe existir un enlace de comunicaciones entre ellos. Este enlace se puede implementar de diferentes formas. No vamos a ocuparnos acá de la implementación física del enlace (memoria compartida, bus hardware o red), sino de su implementación lógica. Existen varios métodos para implementar lógicamente un enlace y las operaciones de envío y recepción:

- Comunicación directa o indirecta.
- Comunicación síncrona o asíncrona.
- Almacenamiento en búfer explícito o automático.